

```

import streamlit as st
import speech_recognition as sr
import tempfile
import os
import datetime
import numpy as np
import pandas as pd

# Optional heavy deps for the Explainable AI (SHAP) section.
# Install with: pip install librosa shap scikit-learn soundfile
try:
    import librosa
    import shap
    from sklearn.ensemble import RandomForestClassifier
    XAI_AVAILABLE = True
except ImportError:
    XAI_AVAILABLE = False

# -----
# LANGUAGE SELECTION (must happen first)
# -----
st.set_page_config(page_title="Cognitive Screening Test / แบบทดสอบคัดกรองสภาวะทางปัญญา")

if "lang" not in st.session_state:
    st.session_state["lang"] = None

if st.session_state["lang"] is None:
    st.title("🧠 Cognitive Screening Test / แบบทดสอบคัดกรองสภาวะทางปัญญา")
    st.write("Please choose your language / กรุณาเลือกภาษา")
    col1, col2 = st.columns(2)
    with col1:
        if st.button("🇹🇹 ภาษาไทย", use_container_width=True):
            st.session_state["lang"] = "th"
            st.rerun()
    with col2:
        if st.button("🇬🇧 English", use_container_width=True):
            st.session_state["lang"] = "en"
            st.rerun()
    st.stop()

LANG = st.session_state["lang"] # "th" or "en"

with st.sidebar:
    st.write("🌐 " + ("ภาษา" if LANG == "th" else "Language") + f": **{'ไทย' if L
```

```

if st.button("🔄 " + ("เปลี่ยนภาษา" if LANG == "th" else "Change language")):
    st.session_state["lang"] = None
    st.rerun()

def T(key: str) -> str:
    """Fetch a localized string."""
    val = STR[key]
    return val[0] if LANG == "th" else val[1]

# _____
# STRING TABLE: key -> (thai, english)
# _____

STR = {
    "title": ("🧠 แบบทดสอบคัดกรองสภาวะทางปัญญา (Cognitive Screening Test)",
              "🧠 Cognitive Screening Test"),

    "record_btn": ("🗣️ บันทึกเสียง", "🗣️ Record your answer"),
    "record_btn_answer": ("🗣️ บันทึกคำตอบ", "🗣️ Record your answer"),
    "transcribing": ("กำลังแปลงเสียงเป็นข้อความ...", "Transcribing..."),
    "transcribe_error": ("ไม่สามารถแปลงเสียงได้ - กรุณาลองอีกครั้ง", "Could not recognise -"),
    "service_error": ("เกิดข้อผิดพลาดของบริการแปลงเสียง: ", "Speech service error: "),
    "you_said": ("คุณพูดว่า:", "You said:"),
    "correct": ("✅ ถูกต้อง!", "✅ Correct!"),
    "incorrect": ("❌ ยังไม่ถูกต้อง", "❌ Not quite."),

    # Section 1
    "sec1_header": ("1. แบบทดสอบความจำคำศัพท์", "1. Word Memory Test"),
    "sec1_instruction": ("กรุณาจดจำคำเหล่านี้:", "Memorize these words:"),

    # Section 1B
    "sec1b_header": ("1B. ทวนคำศัพท์ทันที (Immediate Recall)", "1B. Immediate Recall"),
    "sec1b_instruction": ("พูดทวนคำศัพท์ทั้ง 5 คำที่เพิ่งเห็นด้านบนทันที:",
                          "Repeat back all 5 words you just saw above, right now"),

    # Section 2
    "sec2_header": ("2. แบบทดสอบสมาธิและความจำใช้งาน", "2. Attention Test"),
    "fwd_label": ("**ความจำตัวเลขแบบเรียงไปข้างหน้า** - พูดตัวเลขต่อไปนี้ตามลำดับเดิม:",
                  "**Forward Digit Span** - say these numbers in the same order:"),
    "bwd_label": ("**ความจำตัวเลขแบบย้อนกลับ** - พูดเลข 7-4-2 โดยเรียงย้อนกลับ:",
                  "**Backward Digit Span** - say 7-4-2 in reverse:"),
    "bwd_hint": ("ให้พูดว่า: 2 - 4 - 7", "Say: 2 - 4 - 7"),

    # Section 3
    "sec3_header": ("3. การทวนประโยค (Language Repetition)", "3. Language Repetiti"),
    "sentence_label": ("**ประโยคที่ {n}:**", "**Sentence {n}:**"),

```

```

"match_label": ("ความตรงกัน", "Match"),

# Section 4
"sec4_header": ("4. การคิดเชิงนามธรรม (Abstraction)", "4. Abstraction"),
"abs_correct_prefix": ("🎯 ถูกต้อง!", "🎯 Correct!"),
"abs_incorrect_prefix": ("⚠️ ยังไม่ตรงนัก", "⚠️ Not quite —"),

# Section 5 (orientation to time)
"sec5_header": ("5. การรับรู้เรื่องเวลา (Orientation to Time)", "5. Orientation to Time"),
"ori_day_q": ("**วันนี้วันอะไร?*", "**What day of the week is it today?*"),
"ori_date_q": ("**วันนี้วันที่เท่าไร?*", "**What is today's date (day number)?*"),
"ori_month_q": ("**เดือนนี้คือเดือนอะไร?*", "**What month is it?*"),
"ori_year_q": ("**ปีนี้เป็นปี พ.ศ. อะไร?*", "**What year is it?*"),
"ori_season_q": ("**ตอนนี้เป็นฤดูอะไร?*", "**What season is it now?*"),
"actually_is": ("❌ ที่จริงคือ", "❌ It's actually"),

# Section 6 (orientation to place)
"sec6_header": ("6. การรับรู้เรื่องสถานที่ (Orientation to Place)", "6. Orientation to Place"),
"sec6_caption": ("คำตอบข้อนี้จะถูกบันทึกไว้ ผู้ประเมินสามารถตรวจสอบความถูกต้องด้วยตนเอง",
    "These answers are recorded for a human reviewer to verify,
",
"ori_country_q": ("**ตอนนี้คุณอยู่ประเทศอะไร?*", "**What country are you in right now?*"),
"ori_province_q": ("**ตอนนี้คุณอยู่จังหวัดอะไร?*", "**What state/province are you in right now?*"),
"ori_place_q": ("**ตอนนี้คุณอยู่ที่ไหน (เช่น บ้าน, โรงพยาบาล, คลินิก)?*",
    "**Where are you right now (e.g. home, hospital, clinic)?*"),
"ori_floor_q": ("**ตอนนี้คุณอยู่ชั้นไหนของอาคาร?*", "**What floor of the building are you on?*"),
"ori_city_q": ("**เมืองหรืออำเภอที่คุณอยู่ตอนนี้ชื่ออะไร?*", "**What city or town are you in?*"

# Section 7 (fluency)
"sec7_header": ("7. ความคล่องแคล่วทางภาษา (Verbal Fluency)", "7. Verbal Fluency"),
"fluency_animals_q": ("**บอกชื่อสัตว์ให้ได้มากที่สุดภายใน 1 นาที*",
    "**Name as many animals as you can in 1 minute*"),
"fluency_fruits_q": ("**บอกชื่อผลไม้ให้ได้มากที่สุดภายใน 1 นาที*",
    "**Name as many fruits as you can in 1 minute*"),
"fluency_count": ("🇹🇭 จำนวนคำที่พูดได้โดยประมาณ:", "🇮🇹 Approximate number of words said"),

# Section 8 (calculation)
"sec8_header": ("8. การคำนวณ (Calculation)", "8. Calculation"),
"calc_q": ("**เริ่มจาก 100 แล้วลบ 7 ไปเรื่อยๆ พูดผลลัพธ์ 5 ค่าติดต่อกัน (เช่น 93, 86, 79, 72, 65) *",
    "**Starting at 100, keep subtracting 7 and say 5 results in a row *"),
"calc_result": ("🇹🇭 ถูกต้อง", "🇮🇹 Correct:"),

# Section 9 (naming)
"sec9_header": ("9. การเรียกชื่อสิ่งของ (Naming)", "9. Naming"),
"naming_watch_q": ("**ของใช้ที่ใส่นอกเวลา สวมข้อมือได้ เรียกว่าอะไร?*",
    "**What do you call the object worn on the wrist that tells time?*"),
"naming_pen_q": ("**สิ่งที่ใช้ในการเขียนหนังสือ เรียกว่าอะไร?*",
    "**What do you call the object used for writing?*"),

```

```

"naming_dog_q": ("**สัตว์เลี้ยงที่เห่าและเฝ้าบ้าน เรียกว่าอะไร?*",
                 "**What do you call the pet that barks and guards the house

# Section 10 (functional description)
"sec10_header": ("10. การอธิบายหน้าที่ของสิ่งของ (Functional Description)",
                 "10. Functional Description"),
"func_hammer_q": ("**ค้อนใช้ทำอะไร?*", "**What is a hammer used for?*"),
"func_scissors_q": ("**กรรไกรใช้ทำอะไร?*", "**What are scissors used for?*"),

# Section 11 (proverbs)
"sec11_header": ("11. การอธิบายความหมายสุภาษิต (Proverb Interpretation)",
                 "11. Proverb Interpretation"),
"proverb_water_q": ("**\"น้ำขึ้นให้รีบตัก\" หมายความว่าอย่างไร?*",
                    "**What does \"strike while the iron is hot\" mean?*"),
"proverb_slow_q": ("**\"ช้าๆ ได้พร้าเล่มงาม\" หมายความว่าอย่างไร?*",
                    "**What does \"slow and steady wins the race\" mean?*"),
"proverb_try_again": ("! ลองอธิบายอีกครั้งด้วยคำพูดของคุณเอง", "! Try explaining it

# Section 12 (delayed recall)
"sec12_header": ("12. แบบทดสอบความจำครั้งสุดท้าย (Delayed Recall)", "12. Delayed F
"sec12_instruction": ("พูดคำศัพท์ให้ได้มากที่สุดเท่าที่จำได้จากตอนต้น:",
                      "Say as many words as you can remember from the beginn
"recall_progress": ("ความแม่นยำในการจำคำศัพท์:", "Words Remembered Accuracy:"),
"recall_score": ("🇹🇭 **คะแนน:** จำได้", "🇹🇭 **Score:*"),
"of_words": ("จาก", "out of"),
"words_label": ("คำ", "words recognized."),

# Scoring
"calc_score_btn": ("คำนวณคะแนน (Calculate Score)", "Calculate Score"),
"results_header": ("ผลการทดสอบ", "Results"),
"total_score": ("**คะแนนรวม:", "**Total Score:"),
"good": ("🟢 ผลการทดสอบอยู่ในเกณฑ์ดี (ต้นแบบ/Prototype)", "🟢 Good performance (pr
"mild": ("🟡 มีความกังวลเล็กน้อย (ต้นแบบ/Prototype)", "🟡 Mild concern (prototype)
"review": ("🔴 ควรได้รับการตรวจประเมินเพิ่มเติม (ต้นแบบ/Prototype)", "🔴 Needs review
"disclaimer": ("! นี่ไม่ใช่เครื่องมือวินิจฉัยทางการแพทย์ กรุณาปรึกษาแพทย์ผู้เชี่ยวชาญเพื่อการวินิจฉัยที่
               ! Not a medical diagnosis tool. Please consult a qualified

}

# -----
# CONTENT (word lists, sentences, keywords) PER LANGUAGE
# -----
CONTENT = {
    "th": {
        "words": ["ใบหน้า", "กำมะหยี่", "โบสถ์", "เดซี่", "แดง"],
        "sentences": [
            "ฉันรู้เพียงว่าจอห์นคือคนที่จะช่วยเหลือฉันนี้",
            "แมวมักจะซ่อนตัวใต้โซฟาเวลาที่มีสุนัขอยู่ในห้อง",

```

```

        "เด็กชายวิ่งไปที่สนามเด็กเล่นหลังเลิกเรียนทุกวัน",
        "ฝนตกหนักมากจนถนนหน้าบ้านกลายเป็นแม่น้ำเล็กๆ",
    ],
    "vehicle_kw": ["ยานพาหนะ", "พาหนะ", "เดินทาง", "ล้อ", "ขนส่ง", "ขับ", "นั่ง"],
    "measurement_kw": ["วัด", "เครื่องมือ", "อุปกรณ์", "ตัวเลข", "บอกเวลา", "มาตร"],
    "fruit_kw": ["ผลไม้", "กิน", "รับประทาน", "หวาน", "ผล"],
    "furniture_kw": ["เฟอร์นิเจอร์", "ของใช้", "ไม้", "บ้าน", "เครื่องเรือน"],
    "watch_kw": ["นาฬิกา"],
    "pen_kw": ["ปากกา", "ดินสอ"],
    "dog_kw": ["สุนัข", "หมา"],
    "hammer_kw": ["ตอก", "ตะปู", "ทุบ", "ต่อย"],
    "scissors_kw": ["ตัด", "หนีด"],
    "proverb_water_kw": ["โอกาส", "รีบ", "ฉวย", "ทัน"],
    "proverb_slow_kw": ["ใจเย็น", "ค่อยเป็นค่อยไป", "ระมัดระวัง", "รอบคอบ"],
    "speech_lang_code": "th-TH",
},
"en": {
    "words": ["face", "velvet", "church", "daisy", "red"],
    "sentences": [
        "I only know that John is the one to help today",
        "The cat always hid under the couch when dogs were in the room",
        "The boy runs to the playground after school every day",
        "It rained so hard that the street in front of the house turned into
    ],
    "vehicle_kw": ["vehicle", "transport", "transportation", "wheels", "ride"],
    "measurement_kw": ["measure", "measurement", "tool", "instrument", "scale"],
    "fruit_kw": ["fruit", "eat", "sweet"],
    "furniture_kw": ["furniture", "wood", "sit", "house", "home"],
    "watch_kw": ["watch"],
    "pen_kw": ["pen", "pencil"],
    "dog_kw": ["dog"],
    "hammer_kw": ["hit", "nail", "pound", "strike", "hammer"],
    "scissors_kw": ["cut", "trim", "snip"],
    "proverb_water_kw": ["opportunity", "chance", "act quickly", "seize", "no"],
    "proverb_slow_kw": ["patience", "careful", "steady", "slow", "take your t"],
    "speech_lang_code": "en-US",
},
},
}

C = CONTENT[LANG]

WORDS = C["words"]
SENTENCE_1, SENTENCE_2, SENTENCE_3, SENTENCE_4 = C["sentences"]
FORWARD_SEQUENCE = ["2", "1", "8", "5", "4"]
BACKWARD_SEQUENCE = ["2", "4", "7"]

THAI_WEEKDAYS = ["วันจันทร์", "วันอังคาร", "วันพุธ", "วันพฤหัสบดี", "วันศุกร์", "วันเสาร์", "วันอาทิตย์"]
THAI_MONTHS = ["มกราคม", "กุมภาพันธ์", "มีนาคม", "เมษายน", "พฤษภาคม", "มิถุนายน",

```

"กรกฎาคม", "สิงหาคม", "กันยายน", "ตุลาคม", "พฤศจิกายน", "ธันวาคม"]

```
def current_context():
    now = datetime.datetime.now()
    if LANG == "th":
        weekday = THAI_WEEKDAYS[now.weekday()]
        month = THAI_MONTHS[now.month - 1]
        year = now.year + 543
        if now.month in (3, 4, 5, 6):
            season = "ฤดูร้อน"
        elif now.month in (7, 8, 9, 10):
            season = "ฤดูฝน"
        else:
            season = "ฤดูหนาว"
    else:
        weekday = now.strftime("%A")
        month = now.strftime("%B")
        year = now.year
        if now.month in (12, 1, 2):
            season = "winter"
        elif now.month in (3, 4, 5):
            season = "spring"
        elif now.month in (6, 7, 8):
            season = "summer"
        else:
            season = "fall"
    return {"day": now.day, "weekday": weekday, "month": month, "year": year, "season": season}

def transcribe_audio(audio_bytes: bytes) -> str | None:
    with tempfile.NamedTemporaryFile(delete=False, suffix=".wav") as f:
        f.write(audio_bytes)
        audio_path = f.name
    try:
        r = sr.Recognizer()
        with sr.AudioFile(audio_path) as source:
            audio_data = r.record(source)
        return r.recognize_google(audio_data, language=C["speech_lang_code"])
    except sr.UnknownValueError:
        return None
    except sr.RequestError as e:
        st.error(T("service_error") + str(e))
        return None
    finally:
        os.unlink(audio_path)
```

```

def similarity_score(spoken: str, reference: str) -> float:
    matches = 0
    spoken_clean = spoken.lower().replace(" ", "")
    for w in reference.split():
        if w.lower() in spoken_clean:
            matches += 1
    return matches / max(len(reference.split()), 1)

def digits_in_order(text: str, sequence: list) -> bool:
    spoken_digits = [ch for ch in text if ch.isdigit()]
    return spoken_digits == sequence

def contains_any(text: str, keywords: list) -> bool:
    text_l = text.lower()
    return any(k.lower() in text_l for k in keywords)

def voice_input(label: str, key: str):
    audio = st.audio_input(label, key=key)
    if audio is not None:
        audio_bytes = audio.read()
        if audio_bytes != st.session_state.get(f"{key}_bytes"):
            st.session_state[f"{key}_bytes"] = audio_bytes
            with st.spinner(T("transcribing")):
                result = transcribe_audio(audio_bytes)
                st.session_state[f"{key}_text"] = result if result else ""
                if result is None:
                    st.error(T("transcribe_error"))
    return st.session_state.get(f"{key}_text", "")

# -----
# EXPLAINABLE AI (SHAP) HELPERS
# -----
FEATURE_NAMES = [
    "pitch_mean", "pitch_std", "jitter", "shimmer",
    "energy_mean", "zero_crossing_rate", "mfcc_mean", "speech_rate",
]

FEATURE_LABELS = {
    "pitch_mean": ("ระดับเสียงเฉลี่ย (Pitch เฉลี่ย)", "Mean pitch (F0)"),
    "pitch_std": ("ความแปรปรวนของระดับเสียง", "Pitch variability"),
    "jitter": ("ความสั้นของความถี่เสียง (Jitter)", "Jitter (frequency perturbation)"),
    "shimmer": ("ความสั้นของความดังเสียง (Shimmer)", "Shimmer (amplitude perturbation)")
}

```

```

"energy_mean": ("พลังงานเสียงเฉลี่ย", "Mean energy (RMS)"),
"zero_crossing_rate": ("อัตราการตัดผ่านศูนย์ (ความคมของเสียง)", "Zero-crossing rate"),
"mfcc_mean": ("ค่าเฉลี่ย MFCC (ลักษณะเสียงพูด)", "Mean MFCC (timbre)"),
"speech_rate": ("อัตราความเร็วในการพูด", "Speech rate (words/sec)"),
}

```

```

def extract_acoustic_features(audio_bytes: bytes, transcript: str = "") -> dict |
    """Extract simple acoustic features from a recorded voice clip using librosa.
    if not XAI_AVAILABLE:
        return None
    with tempfile.NamedTemporaryFile(delete=False, suffix=".wav") as f:
        f.write(audio_bytes)
        path = f.name
    try:
        y, sr = librosa.load(path, sr=16000)
        duration = max(librosa.get_duration(y=y, sr=sr), 0.01)

        f0, _, _ = librosa.pyin(
            y, fmin=librosa.note_to_hz("C2"), fmax=librosa.note_to_hz("C7")
        )
        f0_voiced = f0[~np.isnan(f0)] if f0 is not None else np.array([])
        pitch_mean = float(np.mean(f0_voiced)) if len(f0_voiced) else 0.0
        pitch_std = float(np.std(f0_voiced)) if len(f0_voiced) else 0.0
        if len(f0_voiced) > 1:
            periods = 1.0 / f0_voiced
            jitter = float(np.mean(np.abs(np.diff(periods))) / np.mean(periods))
        else:
            jitter = 0.0

        rms = librosa.feature.rms(y=y)[0]
        energy_mean = float(np.mean(rms))
        shimmer = float(np.std(rms) / np.mean(rms)) if np.mean(rms) > 0 else 0.0
        zcr = float(np.mean(librosa.feature.zero_crossing_rate(y)[0]))
        mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
        mfcc_mean = float(np.mean(mfcc))
        word_count = len(transcript.split()) if transcript else 0
        speech_rate = word_count / duration

        return {
            "pitch_mean": pitch_mean, "pitch_std": pitch_std, "jitter": jitter,
            "shimmer": shimmer, "energy_mean": energy_mean, "zero_crossing_rate":
            "mfcc_mean": mfcc_mean, "speech_rate": speech_rate,
        }
    except Exception as e:
        st.error(f"Feature extraction failed: {e}")
        return None

```



```

finally:
    os.unlink(path)

@st.cache_resource
def train_demo_model():
    """
    Trains a small RandomForest on SYNTHETIC data as a demonstration model.
    This is NOT trained on real clinical data and NOT validated – it exists purely
    to illustrate how SHAP-based explainability could work once a real labeled
    Thai speech dataset is available (see the data-collection guideline below).
    """
    rng = np.random.default_rng(42)
    n = 300
    # "typical" class: higher pitch stability, faster/more regular speech
    typical = {
        "pitch_mean": rng.normal(180, 20, n // 2),
        "pitch_std": rng.normal(15, 4, n // 2),
        "jitter": rng.normal(0.01, 0.004, n // 2),
        "shimmer": rng.normal(0.05, 0.015, n // 2),
        "energy_mean": rng.normal(0.05, 0.01, n // 2),
        "zero_crossing_rate": rng.normal(0.08, 0.02, n // 2),
        "mfcc_mean": rng.normal(0, 5, n // 2),
        "speech_rate": rng.normal(2.5, 0.4, n // 2),
    }
    # "possible concern" class: more jitter/shimmer, slower & more variable speech
    concern = {
        "pitch_mean": rng.normal(170, 25, n // 2),
        "pitch_std": rng.normal(25, 6, n // 2),
        "jitter": rng.normal(0.025, 0.008, n // 2),
        "shimmer": rng.normal(0.09, 0.02, n // 2),
        "energy_mean": rng.normal(0.035, 0.012, n // 2),
        "zero_crossing_rate": rng.normal(0.07, 0.02, n // 2),
        "mfcc_mean": rng.normal(-2, 5, n // 2),
        "speech_rate": rng.normal(1.6, 0.4, n // 2),
    }
    X = pd.DataFrame({k: np.concatenate([typical[k], concern[k]]) for k in FEATURES})
    y = np.array([0] * (n // 2) + [1] * (n // 2))
    model = RandomForestClassifier(n_estimators=150, max_depth=5, random_state=42)
    model.fit(X, y)
    return model, X

st.title(T("title"))

# _____
# 1. WORD MEMORY

```

```

# -----
st.subheader(T("sec1_header"))
st.write(T("sec1_instruction"))
st.info("  ·  ".join(WORDS))
st.session_state["words"] = WORDS
st.write("----")

# -----
# 1B. IMMEDIATE RECALL
# -----
st.subheader(T("sec1b_header"))
st.write(T("sec1b_instruction"))
immediate = voice_input(T("record_btn"), "immediate_recall")
if immediate:
    st.success(f"{T('you_said')} **{immediate}**")
    found_now = [w for w in WORDS if w.lower() in immediate.lower()]
    st.write(f" {len(found_now)} / {len(WORDS)}")
    st.session_state["immediate_recall_count"] = len(found_now)
st.write("----")

# -----
# 2. ATTENTION TEST
# -----
st.subheader(T("sec2_header"))

st.markdown(T("fwd_label"))
st.info("2 - 1 - 8 - 5 - 4")
fwd = voice_input(T("record_btn"), "fwd")
if fwd:
    st.success(f"{T('you_said')} **{fwd}**")
    fwd_ok = digits_in_order(fwd, FORWARD_SEQUENCE)
    st.write(T("correct") if fwd_ok else T("incorrect"))
    st.session_state["fwd_ok"] = fwd_ok

st.markdown(T("bwd_label"))
st.info(T("bwd_hint"))
bwd = voice_input(T("record_btn"), "bwd")
if bwd:
    st.success(f"{T('you_said')} **{bwd}**")
    bwd_ok = digits_in_order(bwd, BACKWARD_SEQUENCE)
    st.write(T("correct") if bwd_ok else T("incorrect"))
    st.session_state["bwd_ok"] = bwd_ok

st.write("----")

# -----
# 3. LANGUAGE REPETITION

```

```

# -----
st.subheader(T("sec3_header"))

for i, sentence in enumerate([SENTENCE_1, SENTENCE_2, SENTENCE_3, SENTENCE_4], st
    st.markdown(f"{T('sentence_label').format(n=i)} {sentence}")
    ans = voice_input(T("record_btn"), f"lang{i}")
    if ans:
        sc = similarity_score(ans, sentence)
        st.success(f"{T('you_said')} **{ans}**")
        st.progress(sc, text=f"{T('match_label')}: {sc:.0%}")
        st.session_state[f"lang{i}_score"] = sc

st.write("---")

# -----
# 4. ABSTRACTION
# -----
st.subheader(T("sec4_header"))

abstraction_qs = [
    ("**รถไฟกับจักรยานเหมือนกันอย่างไร?**", "**How are a Train and a Bicycle similar?**"),
    ("**นาฬิกากับไม้บรรทัดเหมือนกันอย่างไร?**", "**How are a Watch and a Ruler similar?**"),
    ("**ส้มกับกล้วยเหมือนกันอย่างไร?**", "**How are an Orange and a Banana similar?**"),
    ("**โต๊ะกับเก้าอี้เหมือนกันอย่างไร?**", "**How are a Table and a Chair similar?**"), "
]

for (q_th, q_en), key, keywords in abstraction_qs:
    st.markdown(q_th if LANG == "th" else q_en)
    ans = voice_input(T("record_btn"), f"{key}_widget")
    if ans:
        st.success(f"{T('you_said')} **{ans}**")
        ok = contains_any(ans, keywords)
        if ok:
            st.info(T("abs_correct_prefix"))
        else:
            st.warning(T("abs_incorrect_prefix"))
        st.session_state[f"{key}_correct"] = ok

st.write("---")

# -----
# 5. ORIENTATION TO TIME
# -----
st.subheader(T("sec5_header"))
ctx = current_context()

ori_time_qs = [
    ("ori_day_q", "ori_day_name", lambda a: ctx["weekday"].replace("วัน", "").lower

```

```

        ("ori_date_q", "ori_date", lambda a: str(ctx["day"]) in a, ctx["day"]),
        ("ori_month_q", "ori_month", lambda a: ctx["month"].lower() in a.lower(), ctx
        ("ori_year_q", "ori_year", lambda a: str(ctx["year"]) in a, ctx["year"]),
        ("ori_season_q", "ori_season", lambda a: ctx["season"].replace("ฤดู", "").lowe
    ]
for label_key, key, check_fn, truth in ori_time_qs:
    st.markdown(T(label_key))
    ans = voice_input(T("record_btn_answer"), key)
    if ans:
        st.success(f"{T('you_said')} **{ans}**")
        ok = check_fn(ans)
        st.write(T("correct") if ok else f"{T('actually_is')} {truth}")
        st.session_state[f"{key}_ok"] = ok

st.write("----")

# -----
# 6. ORIENTATION TO PLACE
# -----

st.subheader(T("sec6_header"))
st.caption(T("sec6_caption"))

place_qs = [
    ("ori_country_q", "ori_country"),
    ("ori_province_q", "ori_province"),
    ("ori_place_q", "ori_place"),
    ("ori_floor_q", "ori_floor"),
    ("ori_city_q", "ori_city"),
]
for label_key, key in place_qs:
    st.markdown(T(label_key))
    ans = voice_input(T("record_btn_answer"), key)
    if ans:
        st.success(f"{T('you_said')} **{ans}**")
        st.session_state[f"{key}_answered"] = True

st.write("----")

# -----
# 7. VERBAL FLUENCY
# -----

st.subheader(T("sec7_header"))

st.markdown(T("fluency_animals_q"))
fluency_animals = voice_input(T("record_btn"), "fluency_animals")
if fluency_animals:
    st.success(f"{T('you_said')} **{fluency_animals}**")

```

```

        words_count = len(set(fluency_animals.lower().split()))
        st.write(f"{T('fluency_count')} {words_count}")
        st.session_state["fluency_animals_count"] = words_count

st.markdown(T("fluency_fruits_q"))
fluency_fruits = voice_input(T("record_btn"), "fluency_fruits")
if fluency_fruits:
    st.success(f"{T('you_said')} **{fluency_fruits}**")
    words_count = len(set(fluency_fruits.lower().split()))
    st.write(f"{T('fluency_count')} {words_count}")
    st.session_state["fluency_fruits_count"] = words_count

st.write("---")

# -----
# 8. CALCULATION
# -----
st.subheader(T("sec8_header"))
st.markdown(T("calc_q"))
calc = voice_input(T("record_btn"), "calc_serial7")
if calc:
    st.success(f"{T('you_said')} **{calc}**")
    expected = ["93", "86", "79", "72", "65"]
    spoken_numbers = [tok for tok in calc.replace(",", " ").split() if tok.isdigit()]
    correct_count = sum(1 for e in expected if e in spoken_numbers)
    st.write(f"{T('calc_result')} {correct_count} / {len(expected)}")
    st.session_state["calc_correct_count"] = correct_count

st.write("---")

# -----
# 9. NAMING
# -----
st.subheader(T("sec9_header"))

naming_qs = [
    ("naming_watch_q", "naming_watch", C["watch_kw"]),
    ("naming_pen_q", "naming_pen", C["pen_kw"]),
    ("naming_dog_q", "naming_dog", C["dog_kw"]),
]

for label_key, key, keywords in naming_qs:
    st.markdown(T(label_key))
    ans = voice_input(T("record_btn_answer"), key)
    if ans:
        st.success(f"{T('you_said')} **{ans}**")
        ok = contains_any(ans, keywords)
        st.write(T("correct") if ok else T("incorrect"))

```

```

        st.session_state[f"{key}_ok"] = ok

st.write("----")

# -----
# 10. FUNCTIONAL DESCRIPTION
# -----

st.subheader(T("sec10_header"))

func_qs = [
    ("func_hammer_q", "func_hammer", C["hammer_kw"]),
    ("func_scissors_q", "func_scissors", C["scissors_kw"]),
]

for label_key, key, keywords in func_qs:
    st.markdown(T(label_key))
    ans = voice_input(T("record_btn_answer"), key)
    if ans:
        st.success(f"{T('you_said')} **{ans}**")
        ok = contains_any(ans, keywords)
        st.write(T("correct") if ok else T("incorrect"))
        st.session_state[f"{key}_ok"] = ok

st.write("----")

# -----
# 11. PROVERB INTERPRETATION
# -----

st.subheader(T("sec11_header"))

proverb_qs = [
    ("proverb_water_q", "proverb_water", C["proverb_water_kw"]),
    ("proverb_slow_q", "proverb_slow", C["proverb_slow_kw"]),
]

for label_key, key, keywords in proverb_qs:
    st.markdown(T(label_key))
    ans = voice_input(T("record_btn_answer"), key)
    if ans:
        st.success(f"{T('you_said')} **{ans}**")
        ok = contains_any(ans, keywords)
        st.write(T("correct") if ok else T("proverb_try_again"))
        st.session_state[f"{key}_ok"] = ok

st.write("----")

# -----
# 12. DELAYED RECALL
# -----

```

```

st.subheader(T("sec12_header"))
st.write(T("sec12_instruction"))

recall = voice_input(T("record_btn"), "recall_widget")
if recall:
    st.success(f"{T('you_said')} {recall}")
    spoken_text = recall.lower()
    words_found = [word for word in WORDS if word.lower() in spoken_text]
    score_recall = len(words_found)
    total_words = len(WORDS)
    pct = score_recall / total_words
    st.progress(pct, text=f"{T('recall_progress')} {pct:.0%}")
    st.write(f"{T('recall_score')} {score_recall} {T('of_words')} {total_words} {
    st.session_state["recall_score_count"] = score_recall

st.write("---")

# -----
# 13. EXPLAINABLE AI – SHAP FEATURE ANALYSIS
# -----
st.subheader(
    "13. วิเคราะห์คุณลักษณะเสียงด้วย Explainable AI (SHAP)"
    if LANG == "th" else
    "13. Explainable AI: Which Voice Features Drive the Result (SHAP)"
)
st.caption(
    "⚠️ โมเดลนี้เป็นเพียงต้นแบบสาธิต ฝึกด้วยข้อมูลสังเคราะห์ (synthetic data) ไม่ใช่ข้อมูลผู้ป่วยจริง "
    "ผลลัพธ์นี้ใช้เพื่อสาธิตแนวทางการอธิบายผลเท่านั้น ไม่ใช่การวินิจฉัย"
    if LANG == "th" else
    "⚠️ This model is only a demonstration prototype trained on synthetic data, n
    "data. Results illustrate how explainability could work – this is not a diagn
)

if not XAI_AVAILABLE:
    st.warning(
        "ต้องติดตั้งไลบรารีเพิ่มเติมก่อนใช้งานส่วนนี้: `pip install librosa shap scikit-learn so
        if LANG == "th" else
        "Install extra libraries to use this section: `pip install librosa shap s
    )
else:
    # Prefer the delayed-recall clip; fall back to any other recorded clip in thi
    candidate_keys = ["recall_widget", "immediate_recall", "lang1", "lang2", "fwd
    chosen_key, chosen_bytes, chosen_text = None, None, ""
    for k in candidate_keys:
        b = st.session_state.get(f"{k}_bytes")
        if b:
            chosen_key, chosen_bytes = k, b

```

```

        chosen_text = st.session_state.get(f"{k}_text", "")
        break

if chosen_bytes is None:
    st.info(
        "บันทึกคำตอบด้วยเสียงอย่างน้อยหนึ่งข้อด้านบนก่อน จึงจะวิเคราะห์ได้"
        if LANG == "th" else
        "Record at least one voice answer above before running this analysis."
    )
else:
    if st.button(
        "🔍 วิเคราะห์คุณลักษณะเสียง (Run SHAP Analysis)"
        if LANG == "th" else
        "🔍 Run SHAP Analysis"
    ):
        with st.spinner("กำลังสกัดคุณลักษณะเสียงและคำนวณ SHAP..." if LANG == "th" else
            feats = extract_acoustic_features(chosen_bytes, chosen_text)
            if feats is not None:
                model, X_train = train_demo_model()
                x_user = pd.DataFrame([feats])[FEATURE_NAMES]
                proba = model.predict_proba(x_user)[0][1]

                explainer = shap.TreeExplainer(model)
                sv = explainer.shap_values(x_user)
                # sv can be a list (per-class) or a single array depending on
                if isinstance(sv, list):
                    shap_vals = sv[1][0]
                else:
                    shap_vals = sv[0]

                label_idx = 0 if LANG == "th" else 1
                labels = [FEATURE_LABELS[f][label_idx] for f in FEATURE_NAMES]
                shap_series = pd.Series(shap_vals, index=labels).sort_values(

                st.write(
                    f"**ความน่าจะเป็นของกลุ่ม 'ควรตรวจเพิ่มเติม' (ต้นแบบสาธิต):** {proba:
                    if LANG == "th" else
                    f"**Model's estimated probability of the 'possible concer
                )
                st.bar_chart(shap_series)
                st.caption(
                    "แท่งที่ยื่นไปทางขวา = คุณลักษณะนั้นดันผลไปทาง 'ควรตรวจเพิ่มเติม' / "
                    "แท่งที่ยื่นไปทางซ้าย = ดันผลไปทาง 'ปกติ' – ยิ่งแท่งยาว ยิ่งมีผลมาก"
                    if LANG == "th" else
                    "Bars pointing right push the result toward 'possible con
                    "left push toward 'typical'. Longer bars = more influence
                )

```



```

        with st.expander("ดูค่าคุณลักษณะดิบ (Raw feature values)":
            st.dataframe(pd.DataFrame([feats]).T.rename(columns={0: "

st.write("---")

# -----
# 14. GUIDELINE: COLLECTING A THAI SPEECH DATASET FOR FUTURE RESEARCH
# -----
st.subheader(
    "14. แนวทางการเก็บชุดข้อมูลเสียงพูดภาษาไทยสำหรับงานวิจัยในอนาคต"
    if LANG == "th" else
    "14. Guideline: Collecting a Thai Speech Dataset for Future Research"
)

with st.expander(
    "อ่านแนวทางฉบับเต็ม" if LANG == "th" else "Read the full guideline", expanded=False
):
    if LANG == "th":
        st.markdown("""
**1. จริยธรรมและความยินยอม**
- ขอความยินยอมที่เป็นลายลักษณ์อักษร (informed consent) จากผู้เข้าร่วมหรือผู้ดูแลตามกฎหมาย
- ผ่านการรับรองจากคณะกรรมการจริยธรรมการวิจัยในมนุษย์ (IRB/EC) ของสถาบัน
- ปฏิบัติตาม พ.ร.บ.คุ้มครองข้อมูลส่วนบุคคล (PDPA) อย่างเคร่งครัด โดยเฉพาะข้อมูลสุขภาพซึ่งจัดเป็นข้อมูลอ่อน

**2. ความหลากหลายของกลุ่มตัวอย่าง**
- ครอบคลุมช่วงอายุ เพศ ระดับการศึกษา และภูมิภาค (ภาคเหนือ ภาคอีสาน ภาคใต้ ภาคกลาง) เพื่อให้ครอบคลุม
- เก็บทั้งกลุ่มปกติและกลุ่มที่ได้รับการวินิจฉัยแล้ว (เช่น MCI, Alzheimer's) โดยแพทย์ผู้เชี่ยวชาญ เพื่อใช้เป็น
- ควบคุมสัดส่วนเพศ/อายุระหว่างกลุ่มให้ใกล้เคียงกัน (matched design) เพื่อลด confounding

**3. โพรโตคอลการบันทึกเสียง**
- ใช้ไมโครโฟนและอัตราสุ่มสัญญาณเสียง (sampling rate) มาตรฐานเดียวกันทุกจุดเก็บข้อมูล (แนะนำ ≥16kHz)
- บันทึกในสภาพแวดล้อมที่ควบคุมเสียงรบกวนพื้นหลัง หรืออย่างน้อยบันทึกระดับเสียงรบกวนไว้เป็น metadata
- ออกแบบชุดงาน (task) ให้หลากหลาย เช่น การอ่านออกเสียง การพูดต่อเนื่องแบบอิสระ (spontaneous speech)

**4. การติดป้ายกำกับข้อมูล (Labeling)**
- เชื่อมโยงกับผลตรวจทางคลินิกมาตรฐาน เช่น MMSE, MoCA, CDR ที่ประเมินโดยแพทย์
- บันทึกวันที่ประเมิน และระยะเวลาห่างจากการบันทึกเสียง เพื่อความถูกต้องของ label
- พิจารณาการเก็บข้อมูลระยะยาว (longitudinal) เพื่อดูการเปลี่ยนแปลงของเสียงพูดตามระยะเวลา

**5. การจัดการและความเป็นส่วนตัวของข้อมูล**
- จัดเก็บไฟล์เสียงและข้อมูลระบุตัวตนแยกจากกัน (de-identification) พร้อมระบบเข้ารหัส
- กำหนดสิทธิ์การเข้าถึงข้อมูลเฉพาะที่วิจัยที่เกี่ยวข้อง และมีบันทึกการเข้าถึง (access log)
- วางแผนระยะเวลาการเก็บรักษาและการทำลายข้อมูลตามนโยบายของสถาบันและกฎหมาย

**6. คุณภาพและ Metadata**
- บันทึก metadata ครบถ้วน เช่น อุปกรณ์บันทึก อายุ เพศ ระดับการศึกษา สำเนียง โรคประจำตัวที่เกี่ยวข้อง ยา
- ตรวจสอบคุณภาพเสียง (SNR, clipping) ก่อนนำเข้าชุดข้อมูลหลัก

```

```

- เตรียมชุดข้อมูลสำหรับ train/validation/test แยกตามผู้พูด (speaker-independent split) เพื่อ
    """)
    else:
        st.markdown("""
**1. Ethics and Consent**
- Obtain written informed consent from participants (or legal guardians where app
- Secure approval from an Institutional Review Board / Ethics Committee
- Comply with Thailand's PDPA (Personal Data Protection Act), treating health dat

**2. Sample Diversity**
- Cover a range of ages, genders, education levels, and regions (North, Northeast
- Include both cognitively typical participants and clinically diagnosed particip
- Match age/gender distribution across groups where possible to reduce confoundin

**3. Recording Protocol**
- Use a consistent microphone setup and sampling rate across all collection sites
- Record in an environment with controlled background noise, or log ambient noise
- Design varied elicitation tasks: reading aloud, spontaneous speech, sentence re

**4. Labeling**
- Link each recording to standardized clinical assessments (MMSE, MoCA, CDR) scor
- Record the assessment date and its gap from the recording date for label accura
- Consider longitudinal collection to track how speech changes over time

**5. Data Management and Privacy**
- Store audio separately from identifying information (de-identification), with e
- Restrict access to the research team only, with an access log
- Define a retention and deletion policy in line with institutional and legal req

**6. Quality and Metadata**
- Record complete metadata: recording device, age, gender, education, dialect/acc
- Check audio quality (SNR, clipping) before inclusion in the main dataset
- Use a speaker-independent train/validation/test split to prevent data leakage
    """)

st.write("----")

# _____
# SCORING
# _____

if st.button(T("calc_score_btn")):
    score = 0
    details = []

    imm = st.session_state.get("immediate_recall_count", 0)
    score += imm
    details.append(f"

```

```

if st.session_state.get("fwd_ok"):
    score += 1
    details.append(f"✅ {T('fwd_label')}")
else:
    details.append(f"❌ {T('fwd_label')}")

if st.session_state.get("bwd_ok"):
    score += 1
    details.append(f"✅ {T('bwd_label')}")
else:
    details.append(f"❌ {T('bwd_label')}")

for i in range(1, 5):
    s = st.session_state.get(f"lang{i}_score", 0.0)
    label = T("sentence_label").format(n=i)
    if s >= 0.6:
        score += 1
        details.append(f"✅ {label} {s:.0%}")
    else:
        details.append(f"❌ {label} {s:.0%}")

for i in range(1, 5):
    if st.session_state.get(f"abs{i}_correct"):
        score += 1
        details.append(f"✅ {T('sec4_header')} #{i}")
    else:
        details.append(f"❌ {T('sec4_header')} #{i}")

ori_time_keys = ["ori_day_name_ok", "ori_date_ok", "ori_month_ok", "ori_year_"]
for key in ori_time_keys:
    if st.session_state.get(key):
        score += 1
        details.append(f"✅ {T('sec5_header')} ({key})")
    else:
        details.append(f"❌ {T('sec5_header')} ({key})")

place_keys = ["ori_country_answered", "ori_province_answered", "ori_place_ans",
              "ori_floor_answered", "ori_city_answered"]
for key in place_keys:
    if st.session_state.get(key):
        score += 1
        details.append(f"✅ {T('sec6_header')} ({key})")
    else:
        details.append(f"❌ {T('sec6_header')} ({key})")

for key in ["fluency_animals_count", "fluency_fruits_count"]:

```

```

count = st.session_state.get(key, 0)
if count >= 8:
    score += 1
    details.append(f"✅ {T('sec7_header')} ({{key}}): {count}")
else:
    details.append(f"❌ {T('sec7_header')} ({{key}}): {count}")

calc_correct = st.session_state.get("calc_correct_count", 0)
score += calc_correct
details.append(f"🧮 {T('sec8_header')}: {calc_correct}/5")

for key in ["naming_watch_ok", "naming_pen_ok", "naming_dog_ok"]:
    if st.session_state.get(key):
        score += 1
        details.append(f"✅ {T('sec9_header')} ({{key}})")
    else:
        details.append(f"❌ {T('sec9_header')} ({{key}})")

for key in ["func_hammer_ok", "func_scissors_ok"]:
    if st.session_state.get(key):
        score += 1
        details.append(f"✅ {T('sec10_header')} ({{key}})")
    else:
        details.append(f"❌ {T('sec10_header')} ({{key}})")

for key in ["proverb_water_ok", "proverb_slow_ok"]:
    if st.session_state.get(key):
        score += 1
        details.append(f"✅ {T('sec11_header')} ({{key}})")
    else:
        details.append(f"❌ {T('sec11_header')} ({{key}})")

found = st.session_state.get("recall_score_count", 0)
score += found
details.append(f"🧠 {T('sec12_header')}: {found}/{len(WORDS)}")

st.subheader(T("results_header"))
for d in details:
    st.write(d)

max_score = len(WORDS) + 2 + 4 + 4 + 5 + 5 + 2 + 5 + 3 + 2 + 2 + len(WORDS)
st.write(f"{T('total_score')} {score} / {max_score}**")

if score >= int(max_score * 0.7):
    st.success(T("good"))
elif score >= int(max_score * 0.4):
    st.warning(T("mild"))

```

```
else:  
    st.error(T("review"))  
  
st.caption(T("disclaimer"))
```